Structure your React app the way senior engineers do.

Six lenses, six axioms, one stack — a one-page-per-lens playbook of the architectural decisions that separate a React developer from a React architect.

By **Petar Ivanov** · **petarivanov.tech**

HOW JUNIOR & SENIOR ENGINEERS READ A CODEBASE

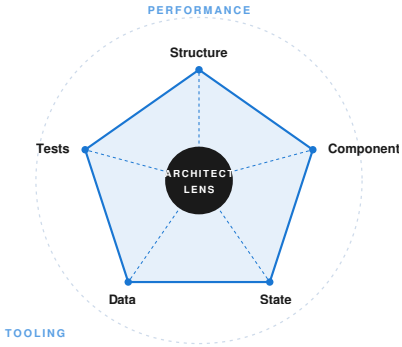
JUNIOR READS

Opens `src/`. Reads file names. Scans top to bottom. Looks for `App.tsx`.

SENIOR READS

Opens `src/`. Reads module boundaries. Traces dependencies. Asks "what owns what?"

THE FIVE LENSES



A senior engineer reads any React codebase through five inner lenses. **Performance** and **tooling** sit on the outer ring — they exist to support the choices made above. Optimize the inner five first; the outer ring follows.

Each page that follows takes one lens deep. Read in order, or jump to the lens you're worst at.

SIX ARCHITECT AXIOMS

01 **Co-locate by domain, not by file type.** A feature is one folder — not files scattered across `components/`, `hooks/`, `utils/`.

02 **State has owners.** Lift state to the closest common ancestor — no higher.

03 **Server state is not client state.** Different lifecycles, different invalidation, different tools.

04 **Composition before memoization.** Restructure the tree first; reach for `memo` / `useMemo` last.

05 **Test what users do, not what code calls.** A test that survives a refactor is a test worth keeping.

06 **Pick tools, not platforms.** Each tool does one thing well and composes with the rest. No lock-in.

STACK BASELINE

BUILD	**`Vite 8 + Rolldown`**	SPAs and libraries — fastest dev loop, smallest config.
	`Next.js`	Content sites and apps that need SSR, SSG, or RSC.
LANGUAGE	**`TypeScript`**	Strict mode, no exceptions past prototype.
PKG MGR	**`pnpm`**	Fast installs, content-addressed store, strict by default.
SCHEMAS	**`Zod`**	Validate at every trust boundary — API responses, forms, env vars.

```
// schemas/User.ts — one line that becomes the contract everywhere downstream
export const User = z.object({ id: z.string(), name: z.string() })
```

> *Each tool does one thing well and composes cleanly with the others. None tries to be a platform. None lock you in.*

Folder structure & module boundaries

A folder structure is a contract. It says *"these files belong together; these don't."* Get it right and the codebase guides every future change. Get it wrong and developers fight the tree before they touch the code.

ANNOTATED FEATURE-BASED TREE

```
src/
├── pages/                    domain modules — one folder per route
│   ├── checkout/
│   │   ├── components/         used only here
│   │   ├── hooks/              // useCheckout, useCartTotal
│   │   ├── services/           // API calls, scoped to checkout
│   │   ├── checkout.test.tsx   test sits next to the file
│   │   └── index.ts            public surface of the module
│   └── profile/
├── common/                   cross-cutting code — the only allowed import target
│   ├── components/             // Button, Modal, Field
│   ├── hooks/                  // useDebounce, useMediaQuery
│   └── lib/                    // fetch client, date utils, zod schemas
├── app/                      routing, providers, layout shell
└── main.tsx
```

Five rules the tree enforces

01 **Group by domain, not by tech.** Pages own their components, hooks, and services.

02 **Modules import from themselves or from `common/` only.** No `checkout` → `profile`.

03 **Common is the shared module.** If two modules need it, lift it to `common/`.

04 **Co-locate close to use.** A subcomponent used by exactly one page lives inside that page.

05 **Wrap third-party libs.** Adapt them in `common/lib` so swapping is one file's problem.

LAYERED VS. FEATURE-BASED VS. HYBRID

LAYERED

Group by tech

Top-level `components/`, `hooks/`, `utils/`. Everyone knows where things go.

Use when: small app, single team, < 30 components.

Breaks at: features bleeding across folders — one change touches five trees.

FEATURE-BASED

Group by domain

One folder per page or module owns its components, hooks, services, and tests.

Use when: mid–large app, multiple teams, change locality matters.

Breaks at: early-stage prototypes — too much ceremony for too little code.

HYBRID

Feature-based + common/

Domains for product surface, layers for shared primitives. Default for production apps.

Use when: almost always — once you have any reusable UI.

Watch for: `common/` turning into a junk drawer.

TRADEOFF — THE COMMON/ TRAP

"Shared" code that's only used by one module is just deferred refactoring. Run a regular pruning pass: anything imported by exactly one feature moves back into that feature.

THE TAKEAWAY

AVOID

A top-level `components/`, `hooks/`, `utils/` tree once features start to span multiple folders — every change becomes a multi-folder hunt.

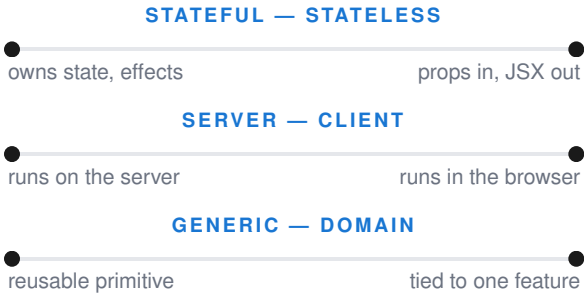
PREFER

A feature-based tree with a single `common/` module — modules import from themselves or from `common/`, never from each other.

Component architecture

Most React components are doing three jobs at once — fetching data, managing state, and rendering UI. Senior engineers split them. The same component gets classified along three independent axes, and each axis decides what stays and what moves out.

THREE AXES OF CLASSIFICATION



Any component is a point on three independent axes. A **generic, stateless, client** component (a `Button`) has the highest reuse potential. A **domain, stateful, server** component is the opposite end — one purpose, one place.

When a component drifts toward the wrong end of an axis, extract: lift state up, push presentation down, or split a domain shell from a generic primitive underneath.

CUSTOM HOOK — THE UNIT OF LOGIC REUSE

```
// hooks/useUserProfile.ts — encapsulates one feature's state machine
export const useUserProfile = (id: string) => {
  const { data, isLoading, error } = useQuery({
    queryKey: ['user', id],
    queryFn: () => api.getUser(id),
  })
  const [isEditing, setEditing] = useState(false)
  return { user: data, isLoading, error, isEditing, setEditing }
}
```

The component just renders the return value. Logic lives in the hook, where it can be tested, shared, and replaced — without dragging JSX along.

Sizing heuristics

- **≤ 5 props.** More than that, group related ones into an object.
- **≤ 150 lines.** Bigger than that, you're hiding two components.
- **One responsibility.** If you describe the component with "and," split it.
- **Top-down readable.** Types → hooks → handlers → JSX. No nested helpers.
- **Early returns.** Guard with loading & error first; happy path last.

COMPOSITION OVER CONFIGURATION

```
// avoid: configuration soup
<Modal title="Confirm" showHeader showFooter
       confirmText="OK" cancelText="Cancel"
       onConfirm={...} onCancel={...} />
```

```
// prefer: composition with slots & children
<Modal>
  <Modal.Header>Confirm</Modal.Header>
  <Modal.Body>Are you sure?</Modal.Body>
  <Modal.Footer>...</Modal.Footer>
</Modal>
```

STYLING · TAILWIND CSS + SHADCN/UI

Tailwind for utility primitives; shadcn/ui for copy-paste, unstyled components you own and can fork. No runtime dependency you can't read.

COMPONENT DEV · STORYBOOK

Develop components in isolation, document variants, run visual regression on every PR. The component lives there before it lives in the app.

THE TAKEAWAY

AVOID

Big components that fetch, decide, and render in 400 lines — impossible to reuse, painful to test, expensive to change.

PREFER

Small components composed from custom hooks (logic) and stateless presentational pieces (UI) — each one testable, replaceable, and named for one job.

State & data fetching

The single insight that retires most state-management debate: **server state is not client state**. Cached, refetchable data from an API has a different lifecycle than a toggle in your UI — and using one tool for both is how you end up with 500 lines of Redux to display a list.

WHERE DOES THIS STATE LIVE?

```
Is it server data?             → TanStack Query       alt: SWR
Is it routing/URL state?       → TanStack Router      alt: Next.js / React Router
Is it search/filter state?     → URL search params
Is it form state?              → React Hook Form + Zod
Shared, frequently changing?   → Zustand              alt: Jotai
Shared, rarely changing?       → React Context
Local to one component?        → useState / useReducer
```

The split that matters

Server state is data your app *doesn't own* — it lives in a database somewhere and may be stale the moment you read it. It needs caching, refetching, and invalidation.

Client state is data your app *does own* — current tab, modal open, draft form. It needs none of that.

Pick the right tool per category and 80% of state-management complexity disappears.

TANSTACK QUERY + ZOD — THE PRODUCTION PATTERN

```ts
// Validate at the boundary; everything downstream gets a typed, guaranteed shape.
const User = z.object({ id: z.string(), name: z.string(), email: z.string().email() })

export const useUser = (id: string) =>
  useQuery({
    queryKey: ['user', id],
    queryFn: async () => User.parse(await api.getUser(id)),
    staleTime: 60_000,        // don't refetch within 1 min
  })
```

API layering — one direction only

```
component → custom hook → service → api client
```

Each layer hides the one below. The component never knows whether the data came from cache, network, or a fixture.

Three caching moves to know

- **Stale-while-revalidate** — show cache, refetch silently. Default on.
- **Optimistic updates** — mutate cache before the server responds, roll back on error.
- **Targeted invalidation** — `invalidateQueries(['user', id])` after a write. Refetches only what changed.

TRADEOFF — CONTEXT VS. ZUSTAND

React Context is fine for theme, locale, or auth user — values that change rarely. Use it for hot, frequently-changing state and every consumer re-renders on every change. Zustand with selectors lets each component subscribe only to the slice it reads.

THE TAKEAWAY

AVOID

Putting fetched API data in a global store and manually keeping it in sync — you're rebuilding a cache layer that already exists, badly.

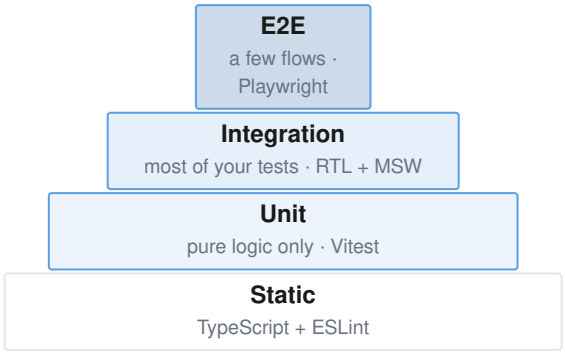
PREFER

Treat server state as a separate concern managed by TanStack Query. Reserve Zustand and Context for true client state.

Testing strategy

Tests are an architecture decision. Pick the wrong shape of pyramid and you'll have 3,000 unit tests that pass while the checkout button is broken. Pick the right shape and a small, fast suite catches almost everything that matters.

THE SHAPE OF A HEALTHY REACT TEST SUITE



What to test where

- **Integration is the bulk.** Render a feature with RTL, mock the network with MSW, assert what the user sees.
- **Unit for pure logic only.** Reducers, validators, calculation helpers.
- **E2E for revenue-critical flows.** Login, checkout, signup. A few is enough.
- **Static is free.** TypeScript and ESLint catch a third of bugs before they're written.
- **Storybook** sits beside the pyramid — component development plus visual regression on every PR.

RECOMMENDED STACK

UNIT / CMP	`Vitest + RTL`	Reuses your Vite config; Jest API where it counts.
NETWORK	`MSW`	Mocks at the network boundary — no `jest.mock('axios')` ever again.
E2E	`Playwright`	Real browsers, parallel by default, network & trace tooling built in.
VISUAL	`Storybook`	Component dev, docs, and visual regression in one place.

INTEGRATION TEST — RTL + MSW

```
test('shows error when password is empty', async () => {
  render(<LoginPage />)
  await userEvent.type(screen.getByLabelText(/email/i), 'pat@x.com')
  await userEvent.click(screen.getByRole('button', { name: /sign in/i }))
  expect(await screen.findByText(/password is required/i)).toBeInTheDocument()
})
```

Description style

- ✓ *"shows error when password is empty"*
- ✓ *"redirects to /home after successful login"*
- ✗ *"calls validatePassword once with empty string"*

Anti-patterns to drop

- **Snapshot tests as default.** They flag noise, not bugs.
- **Coverage chasing.** 100% of trivial branches buys nothing.
- **Implementation tests.** If a refactor breaks the test without breaking the feature, the test was wrong.

THE TAKEAWAY

✗ **AVOID**
A pyramid of unit tests covering implementation details — high green numbers, zero confidence on release day.

✓ **PREFER**
Many integration tests written as user journeys, a few E2E for revenue paths, unit tests reserved for pure logic.

Performance, quality automation & the 10 rules

Performance and tooling are downstream of the five lenses — they support the choices you've already made. Then the closing manifesto: ten heuristics every senior React engineer can recite from memory.

PERFORMANCE — IN THIS ORDER

01 **Move state down.** Local state in a leaf doesn't ripple.

02 **Pass children as children.** Stable reference; parent re-renders skip them.

03 **Stable references.** Don't pass `{}` or `() => {}` as a prop on every render.

04 **useMemo / useCallback.** Last resort. Measure first, then memoize.

05 **React.memo.** Only when props are stable AND render is expensive.

Measure with the React Profiler and the browser's Performance tab — never optimize on a hunch.

Code-splitting taxonomy

- **Route-level.** `React.lazy` per route. Default for SPAs.
- **Component-level.** Heavy modals, editors, charts — load on intent.
- **Library-level.** Lazy-import a 200KB library only when its feature is used.

QUALITY AUTOMATION

FORMAT	**Oxfmt**	Rust, ~10× faster. *alt:* Prettier (mature plugin ecosystem).
LINT	**Oxlint**	Rust, near-zero config. *alt:* ESLint when you need plugin breadth.
HOOKS	**Husky + lint-staged**	Run formatter & linter on staged files before commit.
TYPES	**TypeScript strict**	No `any`, no implicit returns, exact optional props.
SCHEMA	**Zod**	At every external boundary — API, env, forms, persisted state.
CI GATE	**PR gate**	Format · lint · type · test — every commit, no exceptions.

SWITCHING THE DEFAULTS

Use Prettier + ESLint today on plugin-heavy stacks; switch to Oxfmt + Oxlint when speed matters and the rules cover your code.

THE 10 RULES OF SENIOR REACT

01 **Co-locate by domain, not by file type.**

02 **State has owners** — lift to the closest common ancestor, no higher.

03 **Server state is not client state** — use the right tool for each.

04 **Composition before memoization** — restructure the tree first.

05 **≤ 5 props, ≤ 150 lines, one responsibility** per component.

06 **Custom hooks are the unit of logic reuse** — not HOCs, not render props.

07 **Wrap third-party libs in adapters you control** — swapping is one file's problem.

08 **Test what the user does**, not which functions ran.

09 **Measure before you optimize** — the Profiler is free.

10 **Automate every quality check** that doesn't need a human.

> *The best stack is the one you stop noticing — until it saves you.*

Advance your career with best practices and real-world examples on my newsletter — **petarivanov.tech**. The path from React developer to React architect is the path from "I can build this" to "I can decide what to build, how to scale it, and why one architecture beats another."

Created by **Petar Ivanov** · **linkedin.com/in/petarivanovv9** · **petarivanov.tech**